

On the Analysis of the Kuhn–Munkres Algorithm

YANQIAO CHEN, Southern University of Science and Technology, China

1 The Kuhn-Munkres Algorithm

1.1 Key Ideas

Problem Formulation. Let a bipartite graph $G(X \cup Y, E)$ be given, where $|X| = |Y| = n$. Each edge $e \in E$ has a weight c_{ij} . The goal is to find a perfect matching M that minimizes the total cost

$$\sum_{(i,j) \in M} c_{ij}. \quad (1)$$

This problem is equivalent to min-cost max-flow problem, where the source s connected to all vertices in X with edges of capacity 1 and cost 0, and all vertices in Y connected to the sink t with edges of capacity 1 and cost 0. The edges between X and Y have capacity 1 and cost c_{ij} . We run the min-cost max-flow algorithm to find the optimal flow, which corresponds to the min-cost perfect matching.

However, the normal min-cost max-flow algorithms will repeatedly find augmenting paths and update the flow, which can be inefficient, with a time complexity of $O(mnk)$. Here k is the size of matching. In dense graphs, the complexity will be $O(n^4)$ when $m = n^2, k = n$. Kuhn-Munkres algorithm optimizes this process by maintaining a feasible labeling of the vertices and using the concept of "slack" to efficiently find augmenting paths.

The key idea of the Kuhn-Munkres algorithm is to maintain a feasible labeling of the vertices in X and Y called dual solution. We define the cost for the vertices in X as u_i and for the vertices in Y as v_j . The algorithm maintains the invariant that for every edge (i, j) , we have $u_i + v_j \leq c_{ij}$ called dual feasibility. This is known as the complementary slackness condition. If $u_i + v_j = c_{ij}$ for some edge (i, j) , then this edge is said to be tight. We define the reduced cost $\tilde{c}_{ij} = c_{ij} - u_i - v_j \geq 0$. The algorithm will only match on the edges with zero reduced cost, which are the tight edges.

Rationale. The edges with non-zero reduced cost are expensive. If the tight edges form a perfect matching, then the matching must be optimal. This is because, for a tight perfect matching M , we have

$$\sum_{(i,j) \in M} c_{ij} = \sum_{(i,j) \in M} (u_i + v_j) = \sum_i u_i + \sum_j v_j. \quad (2)$$

For any other perfect matching M' , dual feasibility gives

$$\sum_i u_i + \sum_j v_j = \sum_{(i,j) \in M'} (u_i + v_j) \leq \sum_{(i,j) \in M'} c_{ij}. \quad (3)$$

Therefore, by Equations 2 and 3, the tight perfect matching reaches the dual lower bound and is optimal.

1.2 Algorithm Overview

So now we are at how to adjust the labeling. We do the following steps:

- Start with a feasible labeling, for example, $u_i = \min_j c_{ij}$ and $v_j = 0$.

- Expand the matching. If we find an augmenting path, we can update the matching and continue. If not, we need to adjust the labeling to create more tight edges (increase at least one tight edge).
- Repeat the process until the size of the matching is n .

We use an augmented tree and slack array to efficiently find the augmenting path and adjust the labeling. The augmented tree is built on the tight edges, and we maintain a slack array to keep track of the minimum reduced cost for each vertex in Y that is not in the tree. When we need to adjust the labeling, we can quickly find the minimum slack and update the labels accordingly without having to recompute the reduced costs for all edges.

Suppose that we start from a non-matched x_r and try to find an augmented tree. We maintain a set S of vertices in X that are in the tree, and a set T of vertices in Y that are in the tree. Initially, $S = \{x_r\}$ and $T = \emptyset$. We also maintain a slack array, defined in Equation 4, for each vertex y_j not in T .

The slack array. This array is defined by

$$\text{slack}[j] = \min_{i \in S} (c_{ij} - u_i - v_j) \quad (4)$$

for each vertex y_j not in T . It represents the minimum reduced cost of edges from vertices in S to vertex y_j . If $\text{slack}[j] = 0$, it means there is a tight edge from some vertex in S to y_j . If $\text{slack}[j] > 0$, it means we need to adjust the labels to create a tight edge.

When we need to adjust the labels, we find the minimum slack value

$$\delta = \min_{j \notin T} \text{slack}[j]. \quad (5)$$

We then update the labels as follows:

- For each vertex $i \in S$, we set $u_i = u_i + \delta$.
- For each vertex $j \in T$, we set $v_j = v_j - \delta$.
- For each vertex $j \notin T$, we set $\text{slack}[j] = \text{slack}[j] - \delta$.

After this update, the feasibility condition is still satisfied:

- $i \in S, j \in T$: $u_i + v_j$ is unchanged, so the edges between S and T remain tight.
- $i \notin S, j \in T$: $u_i + v_j$ decreases by δ , so the edges between $X \setminus S$ and T remain feasible.
- $i \notin S, j \notin T$: $u_i + v_j$ remains unchanged.
- $i \in S, j \notin T$: We have $\delta = \min_{j \notin T} \min_{i \in S} (c_{ij} - u_i - v_j)$, thus $\delta \leq c_{ij} - u_i - v_j$. After the update, we have $u_i + v_j = (u_i + \delta) + v_j = u_i + v_j + \delta \leq c_{ij}$.

While the property remains satisfied, we have at least one new tight edge from S to $Y \setminus T$. We can then add this edge to the tree and continue the search for an augmenting path. Then finally we can find a perfect matching when the size of the matching is n .

1.3 Pseudocode

The detailed algorithm is shown in Algorithms 1 and 2. We use two arrays `matchX` and `matchY` to store the current matching. If `matchX[i] = j`, then x_i is matched to y_j ; if `matchY[j] = i`, then y_j is matched to x_i . A value of 0 means unmatched.

Here `slackX[j]` records the left vertex that gives the current value of `slack[j]`. Therefore, when a right vertex y_j becomes reachable through a tight edge, `slackX[j]` is also the predecessor of y_j in the alternating tree. This is why the algorithm can reconstruct and flip the augmenting path after it reaches an unmatched right vertex.

Algorithm 1 Kuhn–Munkres Algorithm for Min-Cost Perfect Matching**Require:** Cost matrix $C[1..n][1..n]$ **Ensure:** A min-cost perfect matching represented by matchX

```

1: for  $i \leftarrow 1$  to  $n$  do
2:    $u[i] \leftarrow \min_{1 \leq j \leq n} C[i][j]$ 
3:    $\text{matchX}[i] \leftarrow 0$ 
4: end for
5: for  $j \leftarrow 1$  to  $n$  do
6:    $v[j] \leftarrow 0$ 
7:    $\text{matchY}[j] \leftarrow 0$ 
8: end for
9: for  $r \leftarrow 1$  to  $n$  do
10:   $\text{AUGMENT}(r)$ 
11: end for
12: return matchX

```

2 Analysis of the Algorithm**2.1 Correctness**

We prove the correctness from the primal-dual viewpoint introduced in Section 1.1 and implemented in Algorithms 1 and 2.

Lemma 1. *The labels u and v are always dual feasible, i.e., $u_i + v_j \leq C[i][j]$ for all i, j .*

PROOF. Initially, $u_i = \min_j C[i][j]$ and $v_j = 0$, so $u_i + v_j \leq C[i][j]$. During a label update, let S and T be the two sides of the current alternating tree and let $\delta = \min_{j \notin T} \text{slack}[j]$. For $i \in S, j \in T$, u_i increases by δ and v_j decreases by δ , so their sum is unchanged. For $i \notin S, j \in T$, the sum decreases. For $i \notin S, j \notin T$, the sum is unchanged. The only dangerous case is $i \in S, j \notin T$, but the definition of δ gives $\delta \leq C[i][j] - u_i - v_j$. Thus after the update, $u_i + \delta + v_j \leq C[i][j]$. Hence feasibility is preserved. $\square$

Lemma 2. *The algorithm only matches tight edges.*

PROOF. The alternating tree is expanded only through vertices y_j with $\text{slack}[j] = 0$. By the definition of slack, this means that for $i = \text{slackX}[j]$,

$$C[i][j] - u_i - v_j = 0.$$

Thus the tree edge (x_i, y_j) is tight. The augmentation step only flips edges on this alternating path, so every newly added matching edge is tight. Existing matching edges were tight by induction. Therefore the matching always consists only of tight edges. $\square$

Lemma 3. *Each outer iteration increases the size of the matching by one.*

PROOF. In one outer iteration, the algorithm starts an alternating tree from an unmatched left vertex x_r . If the current equality graph contains an augmenting path, the search finds it and flips the path, increasing the matching size by one. Otherwise, all reachable tight edges have already been explored. The label update chooses the minimum positive slack, so at least one right vertex outside T obtains slack zero and becomes reachable through a new tight edge. Since no vertex is removed from the alternating tree, this process must eventually reach an unmatched right vertex. Flipping the resulting alternating path increases the matching size by one. $\square$

Algorithm 2 AUGMENT(r)

```

1:  $S[i] \leftarrow \text{false}$  for all  $i$ , and  $T[j] \leftarrow \text{false}$  for all  $j$ 
2:  $S[r] \leftarrow \text{true}$ 
3: for  $j \leftarrow 1$  to  $n$  do
4:    $\text{slack}[j] \leftarrow C[r][j] - u[r] - v[j]$ ,  $\text{slackX}[j] \leftarrow r$ 
5: end for
6: while true do
7:   Find  $j_0 \notin T$  with  $\text{slack}[j_0] = 0$ , if such a vertex exists
8:   if no such  $j_0$  exists then
9:      $\delta \leftarrow \min\{\text{slack}[j] : \neg T[j]\}$ 
10:     $u[i] \leftarrow u[i] + \delta$  for every  $i$  with  $S[i] = \text{true}$ 
11:    for  $j \leftarrow 1$  to  $n$  do
12:      if  $T[j]$  then
13:         $v[j] \leftarrow v[j] - \delta$ 
14:      else
15:         $\text{slack}[j] \leftarrow \text{slack}[j] - \delta$ 
16:      end if
17:    end for
18:    Choose  $j_0 \notin T$  with  $\text{slack}[j_0] = 0$ 
19:  end if
20:   $T[j_0] \leftarrow \text{true}$ 
21:  if  $\text{matchY}[j_0] = 0$  then
22:    while  $j_0 \neq 0$  do
23:       $i_0 \leftarrow \text{slackX}[j_0]$ ,  $j_1 \leftarrow \text{matchX}[i_0]$ 
24:       $\text{matchX}[i_0] \leftarrow j_0$ ,  $\text{matchY}[j_0] \leftarrow i_0$ 
25:       $j_0 \leftarrow j_1$ 
26:    end while
27:    return
28:  end if
29:   $i_0 \leftarrow \text{matchY}[j_0]$ ,  $S[i_0] \leftarrow \text{true}$ 
30:  for  $j \leftarrow 1$  to  $n$  with  $\neg T[j]$  do
31:    if  $C[i_0][j] - u[i_0] - v[j] < \text{slack}[j]$  then
32:       $\text{slack}[j] \leftarrow C[i_0][j] - u[i_0] - v[j]$ 
33:       $\text{slackX}[j] \leftarrow i_0$ 
34:    end if
35:  end for
36: end while

```

Theorem 1. *Algorithm 1 returns a min-cost perfect matching.*

PROOF. After n outer iterations, Lemma 3 implies that the algorithm obtains a perfect matching. By Lemma 2, all matched edges are tight. Therefore, for the returned matching M ,

$$\sum_{(i,j) \in M} C[i][j] = \sum_{(i,j) \in M} (u_i + v_j) = \sum_i u_i + \sum_j v_j.$$

For any perfect matching M' , Lemma 1 implies

$$\sum_i u_i + \sum_j v_j = \sum_{(i,j) \in M'} (u_i + v_j) \leq \sum_{(i,j) \in M'} C[i][j].$$

Thus the cost of M equals a lower bound on the cost of every perfect matching. Therefore M is a min-cost perfect matching. $\square$

2.2 Complexity

We analyze the implementation in Algorithms 1 and 2. The initialization of the labels takes $O(n^2)$ time: for each left vertex x_i , we scan all n entries in row i to compute $u_i = \min_j C[i][j]$. Initializing v , `matchX`, and `matchY` takes only $O(n)$ time. Thus the initialization cost is $O(n^2)$.

The main loop performs exactly n augmentations. This is because the matching is initially empty and each successful call to `AUGMENT` increases the matching size by one. It remains to bound the cost of one augmentation.

During a single call to `AUGMENT`, the sets S and T describe one alternating tree. A left vertex is inserted into S only when its matched right vertex has just been inserted into T . Hence each left vertex enters S at most once in this augmentation. Similarly, each right vertex enters T at most once. Therefore the alternating tree has at most n expansion steps.

The slack array in Equation 4 is the reason that one augmentation costs only $O(n^2)$ instead of recomputing all reduced costs from scratch. When a new left vertex x_i is added to S , the algorithm updates

$$\text{slack}[j] = \min_{i \in S} (C[i][j] - u_i - v_j)$$

for all $j \notin T$ by scanning the n right vertices once. This costs $O(n)$. Since at most n left vertices can be added to S , all slack maintenance in one augmentation costs $O(n^2)$.

Label updates also fit into this bound. Whenever there is no zero-slack right vertex outside T , the algorithm computes the value in Equation 5,

$$\delta = \min_{j \notin T} \text{slack}[j]$$

by scanning at most n slack values. Then it updates the labels in S and T and decreases the slack values outside T , again using $O(n)$ time. Each label update makes at least one new right vertex have zero slack, so there can be at most n such updates in one augmentation. Thus all label updates in one augmentation cost $O(n^2)$.

The remaining operations are cheaper. Testing and choosing a zero-slack right vertex costs $O(n)$ per expansion if implemented by a scan, so over at most n expansions this contributes another $O(n^2)$. The final path flipping step follows one alternating path of length at most $2n - 1$, so it costs $O(n)$. Therefore a single augmentation costs

$$O(n^2) + O(n^2) + O(n^2) + O(n) = O(n^2).$$

Since there are n augmentations, the total running time is

$$O(n^2) + n \cdot O(n^2) = O(n^3).$$

This improves over applying a general min-cost max-flow algorithm to the dense bipartite instance, where repeatedly finding shortest augmenting paths may lead to $O(n^4)$ time.

For space complexity, the cost matrix C uses $O(n^2)$ space. The labels u and v , the matching arrays `matchX` and `matchY`, the sets S , T , and the arrays `slack` and `slackX` each use $O(n)$ space. Therefore the total auxiliary space is $O(n)$, and the total space including the input cost matrix is

$$O(n^2) + O(n) = O(n^2).$$

3 From Min-Cost Perfect Matching to Max-Cost Max-Matching

Now suppose we are given a bipartite graph $G = (X \cup Y, E)$ with edge profit w_{ij} , and we want a maximum-cardinality matching with the maximum total profit among all maximum-cardinality matchings. This is the max-cost max-matching problem.

Let $|X| = p$, $|Y| = q$, and $N = p + q$. We construct a complete bipartite graph $H = (L \cup R, L \times R)$ with

$$L = X \cup D_L, \quad R = Y \cup D_R, \quad (6)$$

where $|D_L| = q$ and $|D_R| = p$. Thus $|L| = |R| = N$, so H has perfect matchings. The dummy vertices are used to represent unmatched vertices in the original graph.

Let

$$W = \max\{1, \max_{(i,j) \in E} |w_{ij}|\}, \quad B = (2N + 1)W, \quad M = (N + 1)(B + W). \quad (7)$$

Define the cost matrix C of the min-cost perfect matching instance as follows:

$$C_{ab} = \begin{cases} W - w_{ab}, & a \in X, b \in Y, (a, b) \in E, \\ B + W, & a \in D_L \text{ or } b \in D_R, \\ M, & a \in X, b \in Y, (a, b) \notin E. \end{cases} \quad (8)$$

The first case converts profit maximization into cost minimization. The second case allows the perfect matching to fill all vertices by dummy edges. The third case makes non-edges too expensive to be used.

Consider any perfect matching of H that uses exactly k original edges from E and no non-edge. These k original edges form a valid matching in G . Its total cost is

$$\sum_{(i,j) \in M \cap E} (W - w_{ij}) + (N - k)(B + W) = NW + (N - k)B - \sum_{(i,j) \in M \cap E} w_{ij}. \quad (9)$$

Since $B > (2N)W$ by Equation 7, increasing k by one decreases the term $(N - k)B$ in Equation 9 by more than any possible change in total profit. Therefore the min-cost perfect matching first maximizes the number k of original edges. Among perfect matchings with the same value of k , the terms NW and $(N - k)B$ are fixed, so minimizing the cost is exactly the same as maximizing

$$\sum_{(i,j) \in M \cap E} w_{ij}.$$

Also, by the non-edge case in Equation 8, a perfect matching using a non-edge has cost at least M , while the all-dummy construction has cost $N(B + W) < M$. Thus no optimal solution uses a non-edge. Hence the original edges selected by the min-cost perfect matching in H form a max-cost max-matching in G .

4 Conclusion

In this report, we described the Kuhn–Munkres algorithm for the min-cost perfect matching problem in bipartite graphs. The algorithm maintains dual-feasible labels, searches only on tight edges with zero reduced cost, and updates the labels by the minimum slack when the current equality graph cannot produce an augmenting path. Its correctness follows from dual feasibility and complementary slackness. With the slack array, each augmentation costs $O(n^2)$ time, so the total running time is $O(n^3)$ and the space complexity is $O(n^2)$. Finally, we formulated max-cost max-matching as min-cost perfect matching by adding dummy vertices and choosing costs that first enforce maximum cardinality and then optimize total profit.